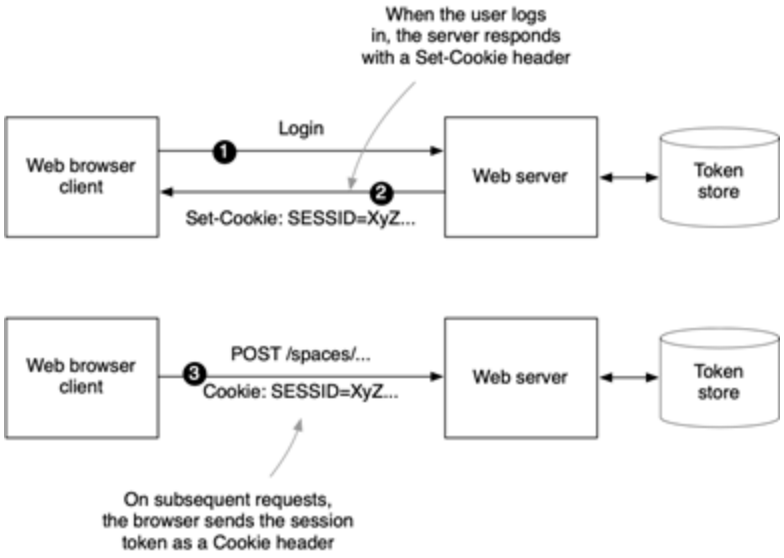


Figure 4.7 In session cookie authentication, after the user logs in the server sends a Set-Cookie header on the response with a random session token. On subsequent requests to the same server, the browser will send the session token back in a Cookie header, which the server can then look up in the token store to access the session state.

Are cookies RESTful?

One of the key principles of the REST architectural style is that interactions between the client and the server should be *stateless*. That is, the server should not store any client-specific state between requests. Cookies appear to violate this principle because the server stores state associated with the cookie for each client. Early uses of session cookies included using them as a place to store temporary state such as a shopping cart of items that have been selected by the user but not yet paid for. These abuses of cookies often broke expected behavior of web pages, such as the behavior of the back button or causing a URL to display differently for one user compared to another.

When used purely to indicate the login state of a user at an API, session cookies are a relatively benign violation of the REST principles, and they have many security attributes that are lost when using other technologies. For example, cookies are associated with a domain and so the browser ensures that they are not accidentally sent to other sites. They can also be marked as Secure, which prevents the cookie being accidentally sent over a non-HTTPS connection where it might be intercepted. I therefore think that cookies still